Moving beyond linearity

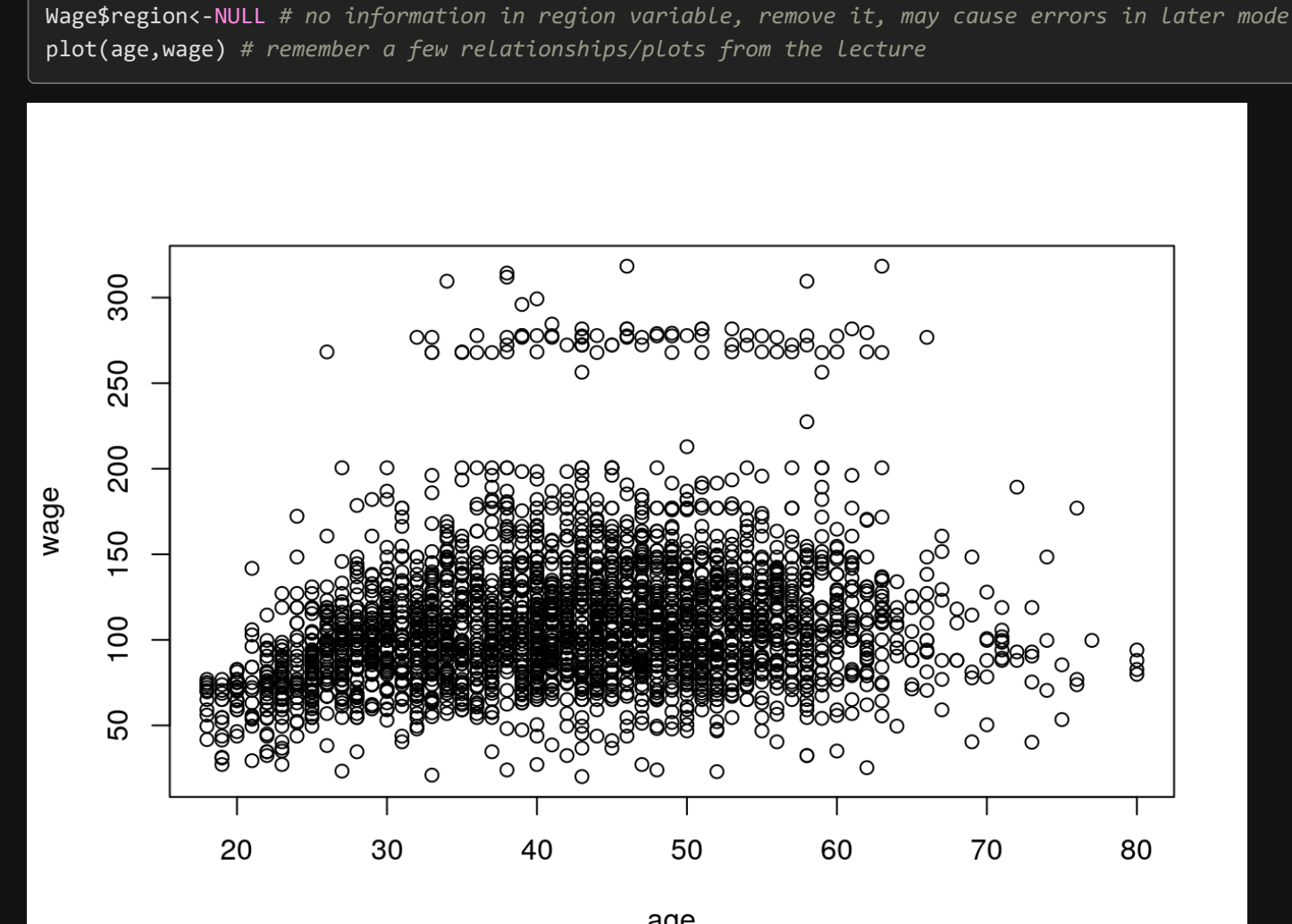
This notebook demonstrates that many of the complex non-linear fitting procedures discussed in the lecture can be easily implemented in R. The notebook uses the same Wage dataset featured in the accompanying lecture on non-linear regression and is directly inspired by ISLR's Chapter 7 Lab: Non-linear Modeling.

Get familiar with the Wage dataset

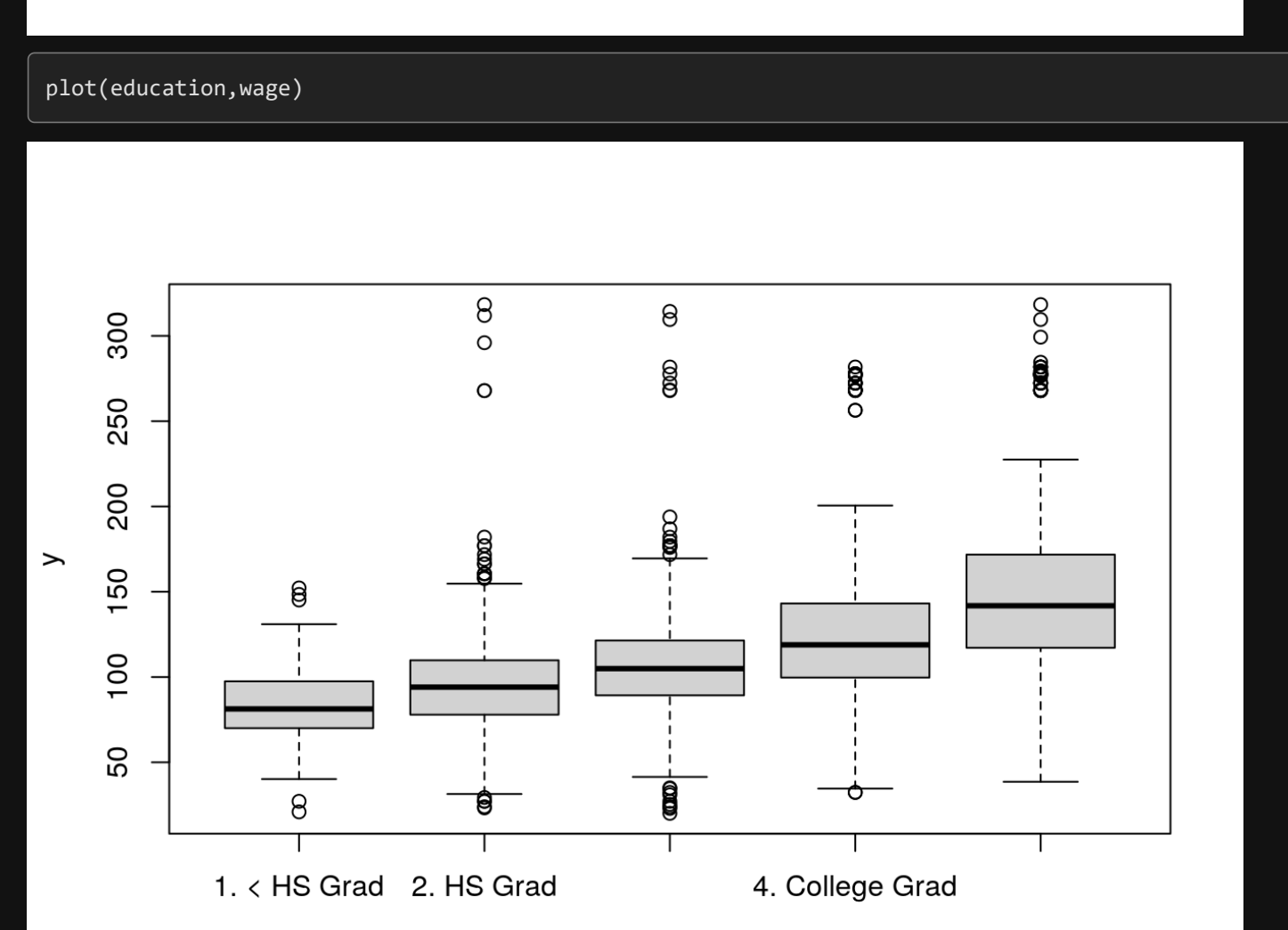
```
help(Wage) # Wage and other data for a group of 3000 male workers in the Mid-Atlantic region
summary(Wage) # learn the structure of the dataset
```

```
##       year           age                     maritl           race     
##  Min.   :2003   Min.   :18.00   1. Never Married: 648   1. White:2480  
##  1st Qu.:2004   1st Qu.:33.75   2. Married      :2074   2. Black: 293  
##  Median :2006   Median :42.00   3. Widowed      :  19   3. Asian: 190  
##  Mean   :2006   Mean   :42.41   4. Divorced     : 204   4. Other:  37  
##  3rd Qu.:2008   3rd Qu.:51.00   5. Separated    :  55                  
##  Max.   :2009   Max.   :80.00                                          
##                                                                        
##               education                     region               jobclass   
##  1. < HS Grad      :268   2. Middle Atlantic   :3000   1. Industrial :1544  
##  2. HS Grad        :971   1. New England       :   0   2. Information:1456  
##  3. Some College   :650   3. East North Central:   0                        
##  4. College Grad   :685   4. West North Central:   0                        
##  5. Advanced Degree:426   5. South Atlantic    :   0                        
##                           6. East South Central:   0                        
##                           (Other)              :   0                        
##             health      health_ins      logwage           wage       
##  1. <=Good     : 858   1. Yes:2083   Min.   :3.000   Min.   : 20.09  
##  2. >=Very Good:2142   2. No : 917   1st Qu.:4.447   1st Qu.: 85.38  
##                                      Median :4.653   Median :104.92  
##                                      Mean   :4.654   Mean   :111.70  
##                                      3rd Qu.:4.857   3rd Qu.:128.68  
##                                      Max.   :5.763   Max.   :318.34  
## 
```

```
Wage$region<-NULL # no information in region variable, remove it, may cause errors in later models
plot(age,wage) # remember a few relationships/plots from the lecture
```



```
plot(education,wage)
```



Polynomial regression

We will model the relationship between age and wage. The plot above suggests that the relationship is not linear. We will begin with polynomial regression. This involves introducing additional higher-order terms into the regression formula. Although the degree of the polynomial is not known, we will work with a degree of 4. Higher degrees are rarely used because the polynomial curve can become excessively flexible. It is important to note that all the models below yield identical predictions; in other words, the fitted values obtained in either case are the same.

```
fit=lm(wage~poly(age,4),data=Wage) # learn the 4th degree polynomial age model with orthogonal polynomials
coef(summary(fit)) # show the model coefs
```

```
##                 Estimate Std. Error    t value     Pr(>|t|)
## (Intercept)    111.70361  0.7287409 153.283015 0.000000e+00
## poly(age, 4)1  447.06785 39.9147851  11.200558 1.484604e-28
## poly(age, 4)2 -478.31581 39.9147851 -11.983424 2.355831e-32
## poly(age, 4)3  125.52169 39.9147851   3.144742 1.678622e-03
## poly(age, 4)4  -77.91118 39.9147851  -1.951938 5.103865e-02
```

```
fit2=lm(wage~poly(age,4,raw=T),data=Wage) # learn the 4th degree polynomial age model with raw polynomials
coef(summary(fit2)) # the coefs and p-values change, later we will see it does not affect the fitted values
```

```
##                             Estimate   Std. Error   t value     Pr(>|t|)
## (Intercept)            -1.841542e+02 6.004038e+01 -3.067172 0.0021802539
## poly(age, 4, raw = T)1  2.124552e+01 5.886748e+00  3.609042 0.0003123618
## poly(age, 4, raw = T)2 -5.638593e-01 2.061083e-01 -2.735743 0.0062606446
## poly(age, 4, raw = T)3  6.810688e-03 3.065931e-03  2.221409 0.0263977518
## poly(age, 4, raw = T)4 -3.203830e-05 1.641359e-05 -1.951938 0.0510386498
```

```
fit2a=lm(wage~age+I(age^2)+I(age^3)+I(age^4),data=Wage) # explicitly calculate the higher-order terms, I inhibits the interp
retation of ^ as a formula crossing/interaction operator, it would make no effect with the only variable age
coef(fit2a) # no change from fit2
```

```
##   (Intercept)           age      I(age^2)      I(age^3)      I(age^4) 
## -1.841542e+02  2.124552e+01 -5.638593e-01  6.810688e-03 -3.203830e-05
```

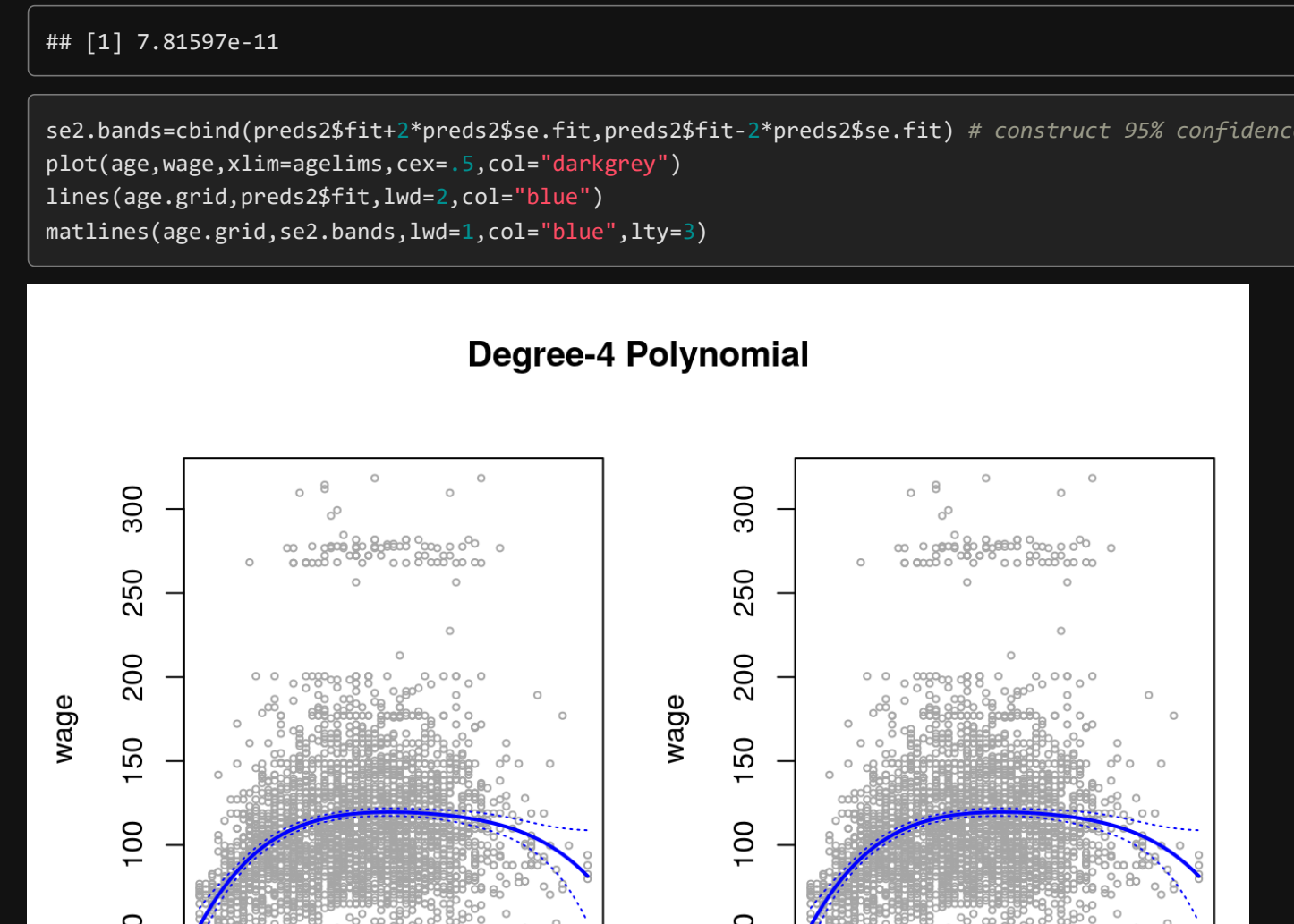
```
fit2b=lm(wage~cbind(age,age^2,age^3,age^4),data=Wage) # explicitly calculate the higher powers, store them into a data.frame
coef(fit2b) # no change from fit2 nor fit2a
```

```
##                         (Intercept) cbind(age, age^2, age^3, age^4)age 
##                       -1.841542e+02                       2.124552e+01 
##     cbind(age, age^2, age^3, age^4)    cbind(age, age^2, age^3, age^4) 
##                       -5.638593e-01                       6.810688e-03 
##     cbind(age, age^2, age^3, age^4) 
##                       -3.203830e-05
```

```
agelims=range(age) # compare all the models created above, create a regular grid of ages to test
age.grid=seq(from=agelims[1],to=agelims[2])
preds=predict(fit,newdata=list(age=age.grid),se=TRUE) # use the orthogonal polynomial model first
se.bands=cbind(preds$fit+2*preds$se.fit,preds$fit-2*preds$se.fit) # construct 95% confidence intervals
par(mfrow=c(1,2),mar=c(4.5,4.5,1,1),oma=c(0,0,4,0))
plot(age,wage,xlim=agelims,cex=.5,col="darkgrey")
title("Degree-4 Polynomial",outer=T)
lines(age.grid,preds$fit,lwd=2,col="blue")
matlines(age.grid,se.bands,lwd=1,col="blue",lty=3)
preds2=predict(fit2,newdata=list(age=age.grid),se=TRUE) # compare with the second fit (we have already found out that 2b and
2c must be identical)
max(abs(preds$fit-preds2$fit)) # there is nearly no difference between the predictions
```

```
## [1] 7.81597e-11
```

```
se2.bands=cbind(preds2$fit+2*preds2$se.fit,preds2$fit-2*preds2$se.fit) # construct 95% confidence intervals
plot(age,wage,xlim=agelims,cex=.5,col="darkgrey")
lines(age.grid,preds2$fit,lwd=2,col="blue")
matlines(age.grid,se2.bands,lwd=1,col="blue",lty=3)
```



Now, we will employ hypothesis testing to decide the optimal degree of the polynomial. We will fit models ranging from linear to a degree-5 polynomial and seek to determine the simplest model that cannot be overcome with a higher degree model.

```
fit.1=lm(wage~age,data=Wage)
fit.2=lm(wage~poly(age,2),data=Wage)
fit.3=lm(wage~poly(age,3),data=Wage)
fit.4=lm(wage~poly(age,4),data=Wage)
fit.5=lm(wage~poly(age,5),data=Wage)
anova(fit.1,fit.2,fit.3,fit.4,fit.5)
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2998	5022216	NA	NA	NA	NA
2	2997	4793430	1	228786.010	143.5931074	2.367734e-32
3	2996	4777674	1	15755.694	9.8887559	1.679213e-03
4	2995	4771604	1	6070.152	3.8098134	5.104623e-02
5	2994	4770322	1	1282.563	0.8049758	3.696820e-01

5 rows

```
coef(summary(fit.5))
```

```
##                 Estimate Std. Error     t value     Pr(>|t|)
## (Intercept)    111.70361  0.7287647 153.2780243 0.000000e+00
## poly(age, 5)1  447.06785 39.9160847  11.2001930 1.491111e-28
## poly(age, 5)2 -478.31581 39.9160847 -11.9830341 2.367734e-32
## poly(age, 5)3  125.52169 39.9160847   3.1446392 1.679213e-03
## poly(age, 5)4  -77.91118 39.9160847  -1.9518743 5.104623e-02
## poly(age, 5)5  -35.81289 39.9160847  -0.8972045 3.696820e-01
```

```
(-11.983)^2 # show that the square of the t-stat for the quadratic model equals the F-stat for the quadratic model taken fro
m ANOVA
```

```
## [1] 143.5923
```

The models must be nested (the set of predictors of a previous model has to be a subset of predictors in its successor model). ANOVA performs a sequence of F-tests, they always compare the simpler model to the more complex model that follows it. The first p-value (<2.2e-16) indicates that a linear fit is not sufficient. The quadratic model seems to be insufficient too (p-value 0.0017). The subsequent p-value is around 0.05 and the last one is large. Consequently, either a cubic or a quartic polynomial appear to reasonably fit the data.

Eventually, notice one of the advantages of orthogonal polynomials. Instead of using the anova() function we could have obtained the same result directly from the degree-5 model. The p-values there match the ANOVA p-values, the square of t-statistics are equal to ANOVA F-statistics. This relationship between the statistics generally holds in simple linear regression only (single independent variable).

In general, the age predictor will be used in combination with other predictors. Let us see the performance of the model that employs education too.

```
fit.1=lm(wage~education+age,data=Wage)
fit.2=lm(wage~education+poly(age,2),data=Wage)
fit.3=lm(wage~education+poly(age,3),data=Wage)
anova(fit.1,fit.2,fit.3)
```

	Res.Df	RSS	Df	Sum of Sq	F	Pr(>F)
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2994	3867992	NA	NA	NA	NA
2	2993	3725395	1	142597.10	114.696898	2.728971e-26
3	2992	3719809	1	5586.66	4.493588	3.410431e-02

3 rows

The conclusion is simple, the cubic age model overcomes the linear and quadratic ones.

Step functions

Step functions allow for locally constant approximation of the dependent variable. The range of age is broken into bins, a different constant is used in each bin. The cut function does the job here, an ordered catagorical variable originates. We propose to split into 4 different bins. The range of the age variable is divided into bins of equal length independently of its actual distribution, the split may miss the actual breakpoints.

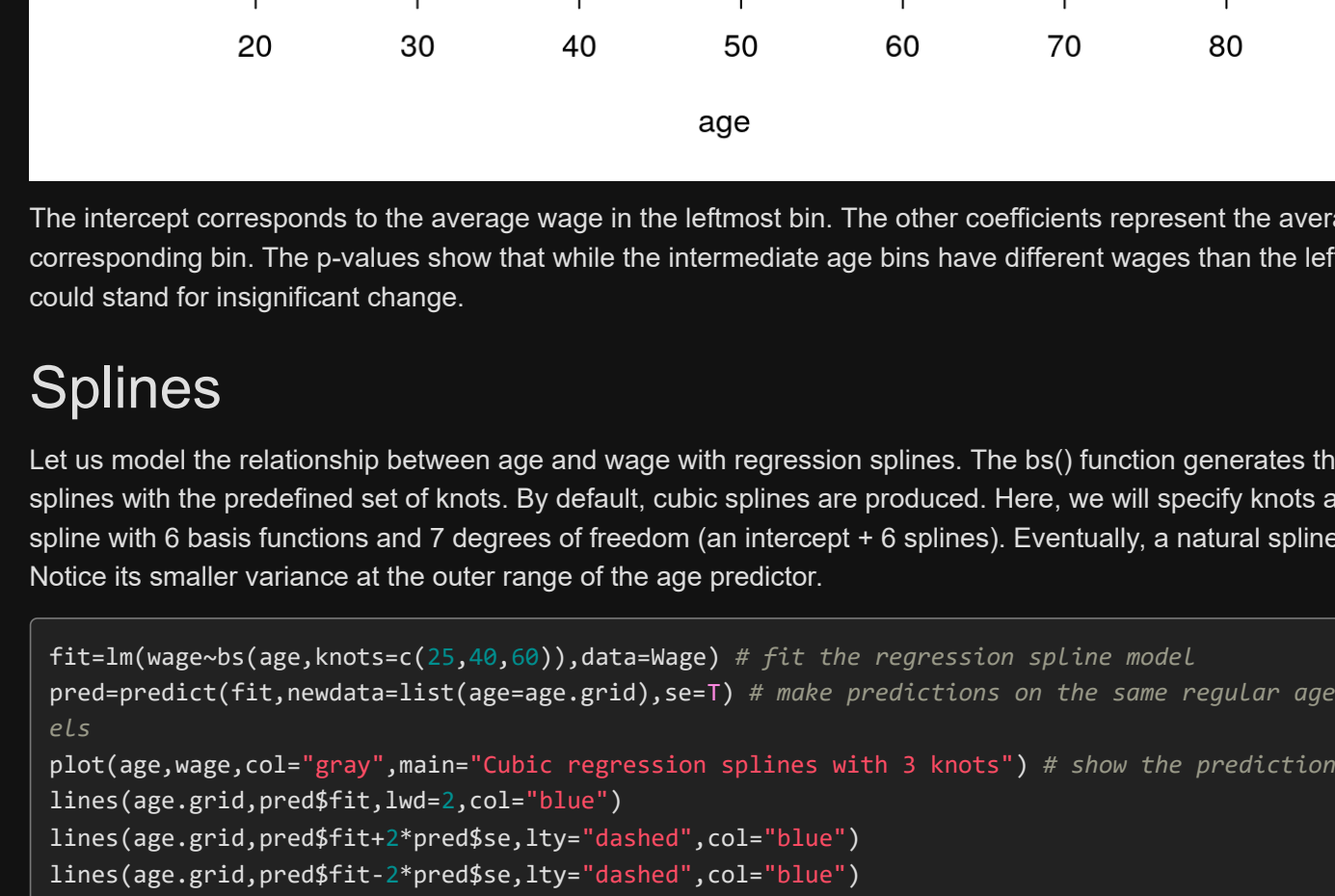
```
table(cut(age,4))
```

```
## 
## (17.9,33.5]   (33.5,49]   (49,64.5] (64.5,80.1] 
##         750        1399         779          72
```

```
fit=lm(wage~cut(age,4),data=Wage)
coef(summary(fit))
```

```
##                         Estimate Std. Error   t value     Pr(>|t|)
## (Intercept)            94.158392   1.476069 63.789970 0.000000e+00
## cut(age, 4)(33.5,49]   24.053491   1.829431 13.148074 1.982315e-38
## cut(age, 4)(49,64.5]   23.664559   2.067958 11.443444 1.040750e-29
## cut(age, 4)(64.5,80.1]  7.640592   4.987424  1.531972 1.256350e-01
```

```
preds=predict(fit,newdata=list(age=age.grid),se=TRUE) # make predictions on the same regular age grid we used for polynomial
models
se.bands=cbind(preds$fit+2*preds$se.fit,preds$fit-2*preds$se.fit) # construct 95% confidence intervals
plot(age,wage,xlim=agelims,cex=.5,col="darkgrey",main="Step regression")
lines(age.grid,preds$fit,lwd=2,col="darkgreen")
matlines(age.grid,se.bands,lwd=1,col="darkgreen",lty=3)
```



The intercept corresponds to the average wage in the leftmost bin. The other coefficients represent the average additional wage in the corresponding bin. The p-values show that while the intermediate age bins have different wages than the leftmost bin, the rightmost bin coefficient could stand for insignificant change.

Splines

Let us model the relationship between age and wage with regression splines. The bs() function generates the entire matrix of basis functions for splines with the predefined set of knots. By default, cubic splines are produced. Here, we will specify knots at ages 25, 40 and 60. This produces a spline with 6 basis functions and 7 degrees of freedom (an intercept + 6 splines). Eventually, a natural spline with 4 degrees of freedom is added. Notice its smaller variance at the outer range of the age predictor.

```
fit=lm(wage~bs(age,knots=c(25,40,60)),data=Wage) # fit the regression spline model
pred=predict(fit,newdata=list(age=age.grid),se=T) # make predictions on the same regular age grid we used for polynomial mod
els
plot(age,wage,col="gray",main="Cubic regression splines with 3 knots") # show the prediction plot
lines(age.grid,pred$fit,lwd=2,col="blue")
lines(age.grid,pred$fit+2*pred$se,lty="dashed",col="blue")
lines(age.grid,pred$fit-2*pred$se,lty="dashed",col="blue")
dim(bs(age,knots=c(25,40,60))) # 6 splines produced
```

```
## [1] 3000    6
```

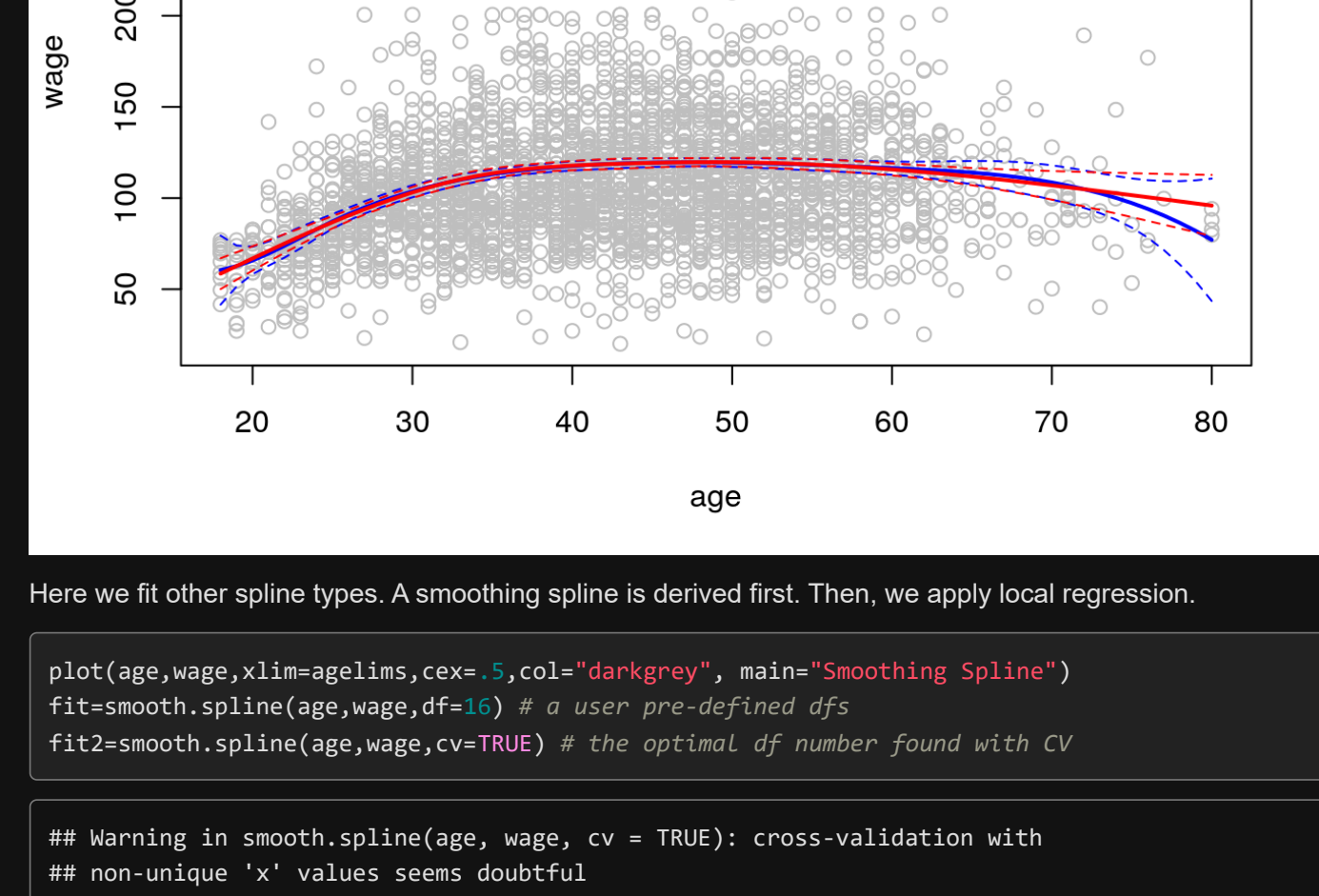
```
dim(bs(age,df=6)) # alternative definition, 6 splines again. knots at uniform quantiles of age
```

```
## [1] 3000    6
```

```
attr(bs(age,df=6),"knots") # show the knots, taken at uniform quantiles (25th, 50th, 75th), their number derived from df
```

```
## [1] 33.75 42.00 51.00
```

```
fit2=lm(wage~ns(age,df=4),data=Wage)
pred2=predict(fit2,newdata=list(age=age.grid),se=T)
lines(age.grid, pred2$fit,col="red",lwd=2)
lines(age.grid,pred2$fit+2*pred2$se,lty="dashed",col="red")
lines(age.grid,pred2$fit-2*pred2$se,lty="dashed",col="red")
legend("topright",legend=c("Cubic spline","Natural cubic spline"),col=c("blue","red"),lty=1,lwd=2,cex=.8)
```



Here we fit other spline types. A smoothing spline is derived first. Then, we apply local regression.

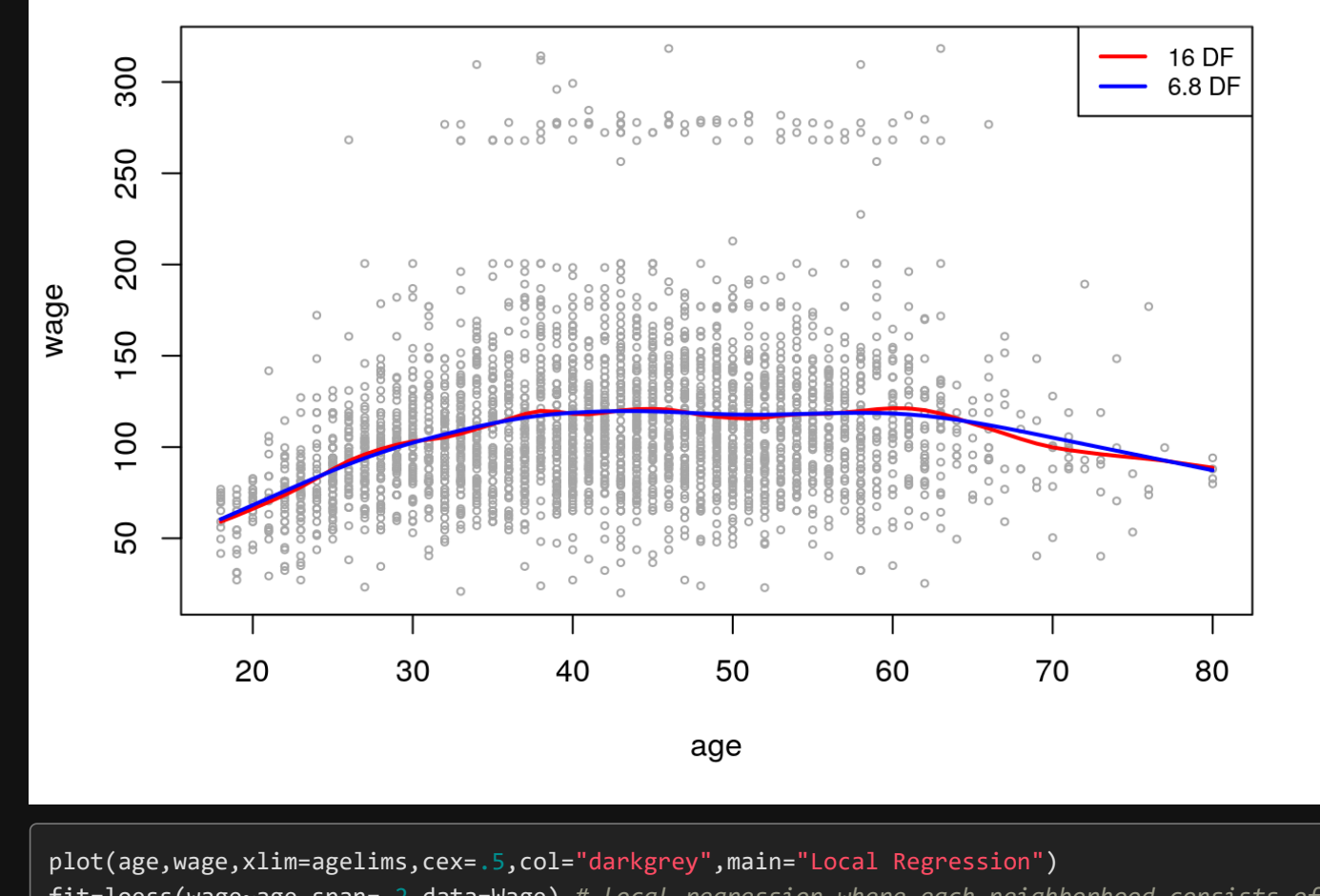
```
plot(age,wage,xlim=agelims,cex=.5,col="darkgrey", main="Smoothing Spline")
fit=smooth.spline(age,wage,df=16) # a user pre-defined dfs
fit2=smooth.spline(age,wage,cv=TRUE) # the optimal df number found with CV
```

```
## Warning in smooth.spline(age, wage, cv = TRUE): cross-validation with
## non-unique 'x' values seems doubtful
```

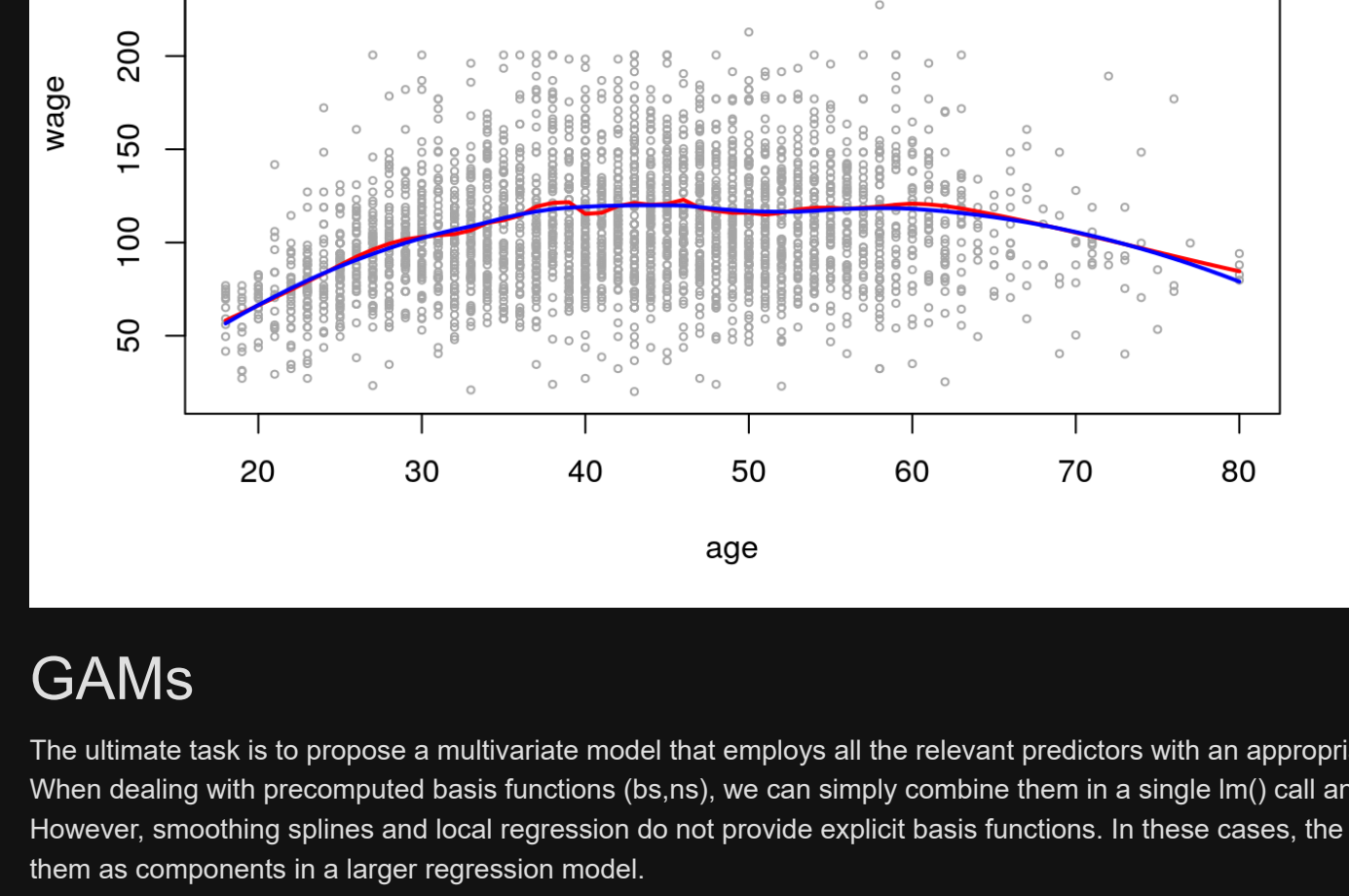
```
fit2$df # the number of dfs found with CV
```

```
## [1] 6.794596
```

```
lines(fit,col="red",lwd=2)
lines(fit2,col="blue",lwd=2)
legend("topright",legend=c("16 DF","6.8 DF"),col=c("red","blue"),lty=1,lwd=2,cex=.8)
```



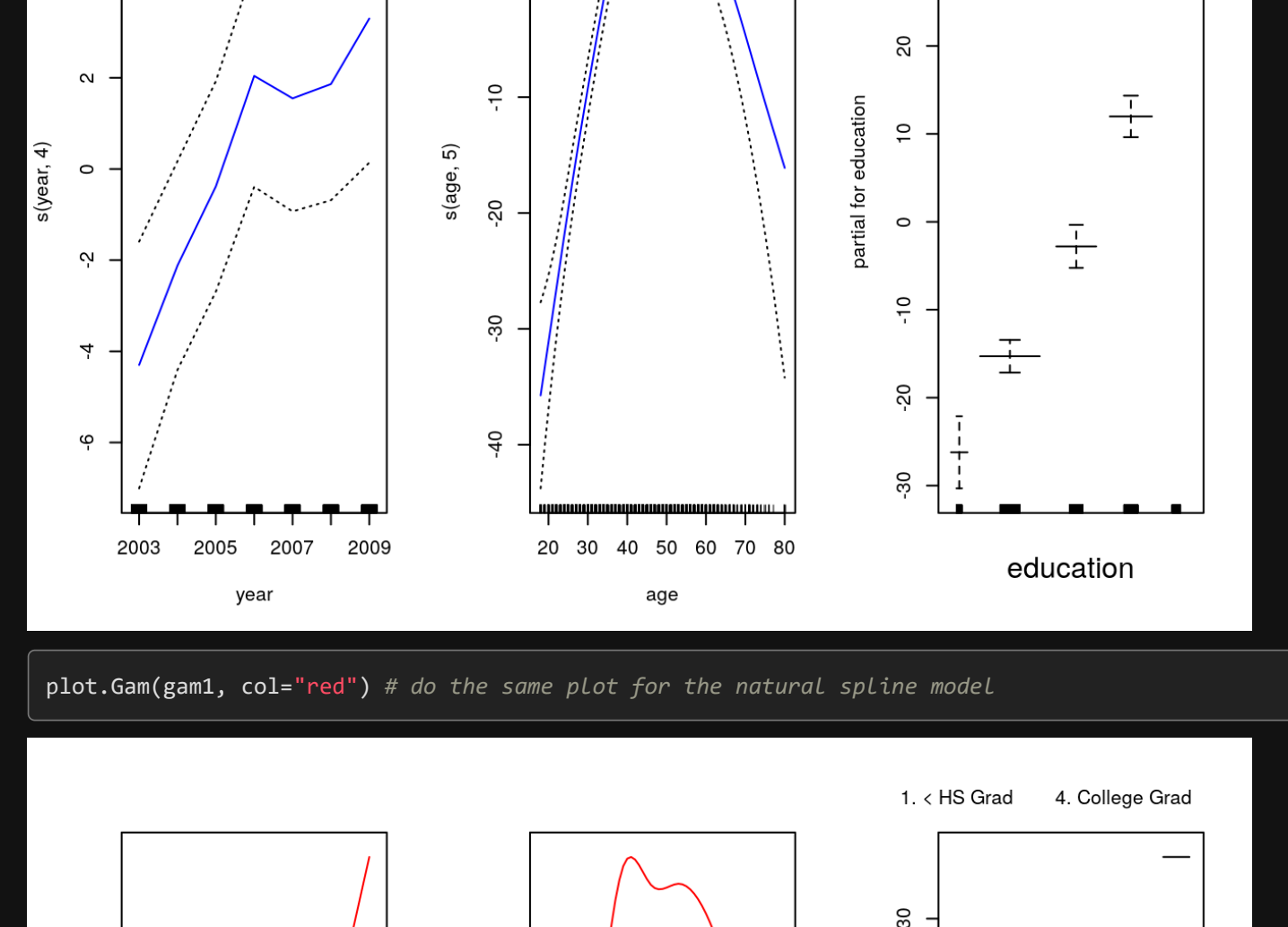
```
plot(age,wage,xlim=agelims,cex=.5,col="darkgrey",main="Local Regression")
fit=loess(wage~age,span=.2,data=Wage) # local regression where each neighborhood consists of 20% of the observations
fit2=loess(wage~age,span=.5,data=Wage) # local regression where each neighborhood consists of 50% of the observations
lines(age.grid,predict(fit,data.frame(age=age.grid)),col="red",lwd=2)
lines(age.grid,predict(fit2,data.frame(age=age.grid)),col="blue",lwd=2)
legend("topright",legend=c("Span=0.2","Span=0.5"),col=c("red","blue"),lty=1,lwd=2,cex=.8)
```



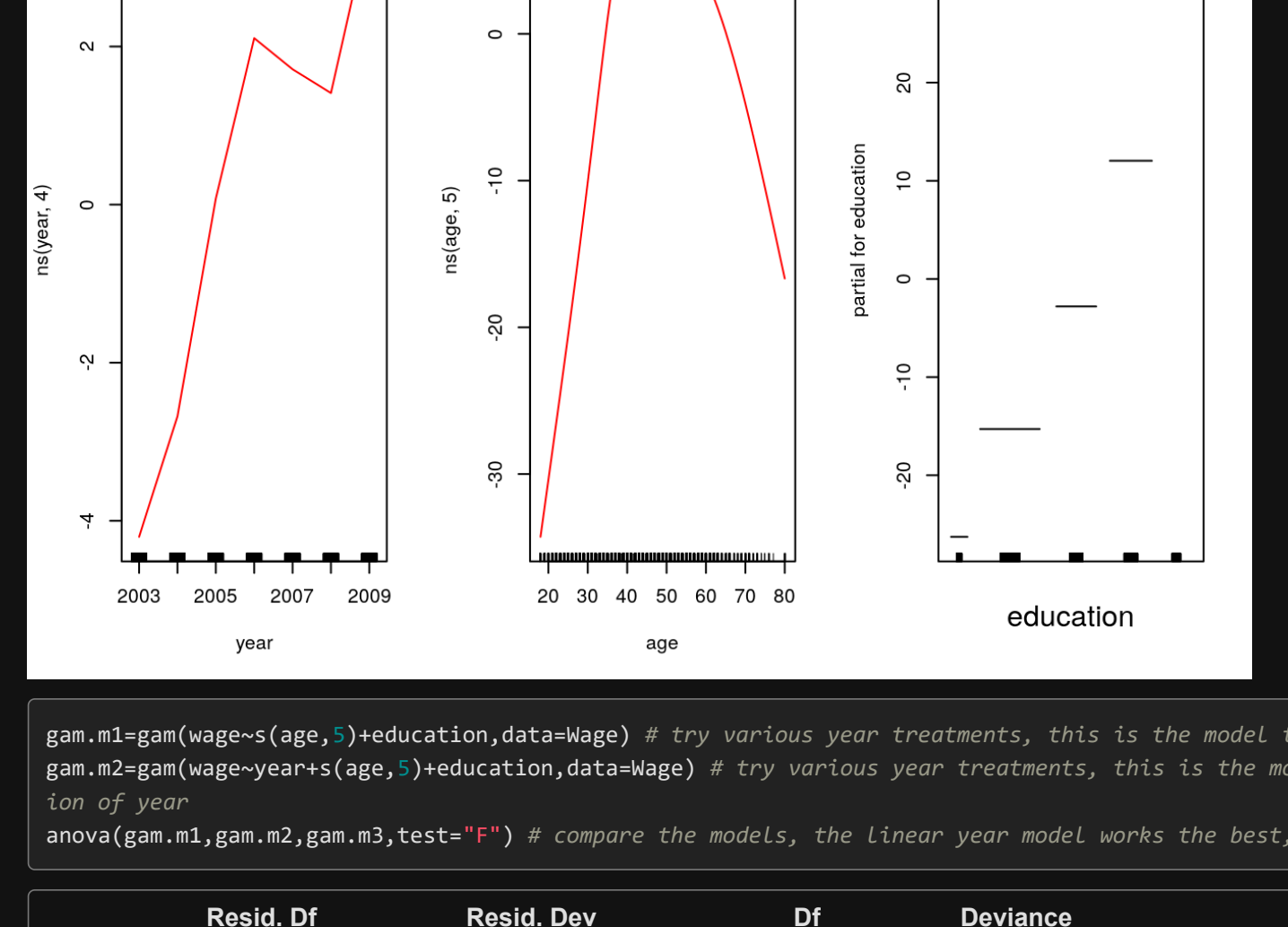
GAMs

The ultimate task is to propose a multivariate model that employs all the relevant predictors with an appropriate choice of their non-linear basis. When dealing with precomputed basis functions (bs,ns), we can simply combine them in a single lm() call and fit the model using least squares. However, smoothing splines and local regression do not provide explicit basis functions. In these cases, the gam library must be used to include them as components in a larger regression model.

```
gam1=lm(wage~ns(year,4)+ns(age,5)+education,data=Wage) # simple linear model when using bs() and ns()
gam.m3=gam(wage~s(year,4)+s(age,5)+education,data=Wage) # gam must be used to compile smoothing splines and/or local regress
ion
par(mfrow=c(1,3))
plot(gam.m3, se=TRUE,col="blue") # show the gam model that uses smoothing splines (education is categorical and converted in
to dummy vars), demonstrate the individual components of the model
```



```
plot.Gam(gam1, col="red") # do the same plot for the natural spline model
```



```
gam.m1=gam(wage~s(age,5)+education,data=Wage) # try various year treatments, this is the model that ignores year
gam.m2=gam(wage~year+s(age,5)+education,data=Wage) # try various year treatments, this is the model that uses a linear funct
ion of year
anova(gam.m1,gam.m2,gam.m3,test="F") # compare the models, the linear year model works the best, m2 is preferred
```

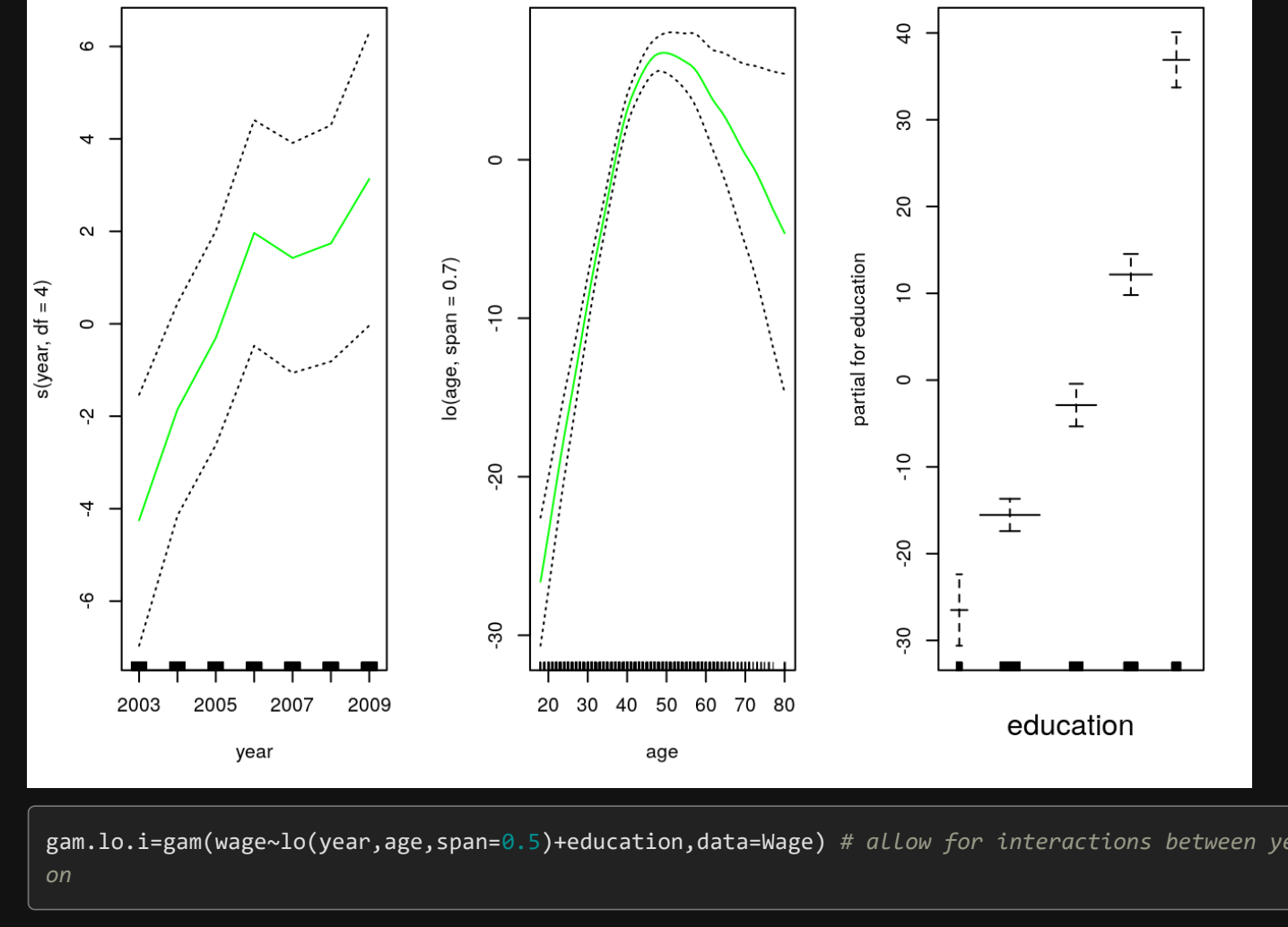
	Resid. Df	Resid. Dev	Df	Deviance	F	Pr(>F)
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	2990	3711731	NA	NA	NA	NA
2	2989	3693842	1.000000	17889.243	14.477130	0.0001447167
3	2986	3689770	2.999989	4071.134	1.098212	0.3485661430

3 rows

```
summary(gam.m3) # a summary of the gam fit, p-values for year and age correspond to a H0 of a linear relationship versus Ha
of a non-linear relationship, a linear function is sufficient for year, however, a non-linear term is requirred for age
```

```
## 
## Call: gam(formula = wage ~ s(year, 4) + s(age, 5) + education, data = Wage)
## Deviance Residuals:
##     Min      1Q  Median      3Q     Max 
## -119.43  -19.70   -3.33   14.17  213.48 
## 
## (Dispersion Parameter for gaussian family taken to be 1235.69)
## 
##     Null Deviance: 5222086 on 2999 degrees of freedom
## Residual Deviance: 3689770 on 2986 degrees of freedom
## AIC: 29887.75 
## 
## Number of Local Scoring Iterations: NA 
## 
## Anova for Parametric Effects
##              Df  Sum Sq Mean Sq F value    Pr(>F)    
## s(year, 4)    1   27162   27162  21.981 2.877e-06 ***
## s(age, 5)     1  195338  195338 158.081 < 2.2e-16 ***
## education     4 1069726  267432 216.423 < 2.2e-16 ***
## Residuals  2986 3689770    1236                      
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## 
## Anova for Nonparametric Effects
##             Npar Df Npar F  Pr(F)    
## (Intercept)                          
## s(year, 4)        3  1.086 0.3537    
## s(age, 5)         4 32.380 <2e-16 ***
## education                            
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
preds=predict(gam.m2,newdata=Wage) # make prediction from gam object
gam.lo=gam(wage~s(year,df=4)+lo(age,span=0.7)+education,data=Wage) # learn a gam with local regression building blocks
plot(gam.lo, se=TRUE, col="green")
```



```
gam.lo.i=gam(wage~lo(year,age,span=0.5)+education,data=Wage) # allow for interactions between year and age in local regressi
on
```

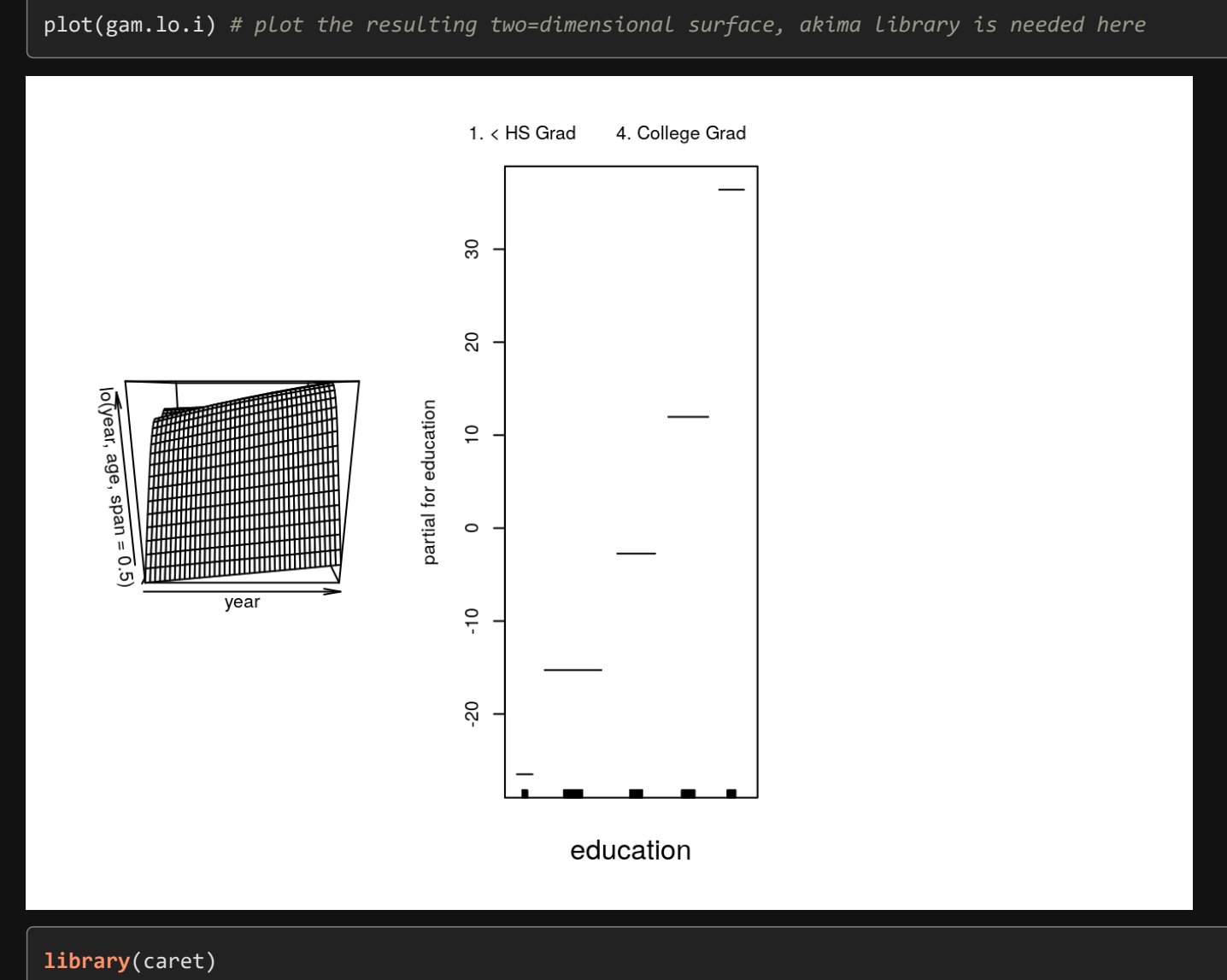
```
## Warning in lo.wam(x, z, wz, fit$smooth, which, fit$smooth.frame, bf.maxit, :
## liv too small.  (Discovered by lowesd)
```

```
## Warning in lo.wam(x, z, wz, fit$smooth, which, fit$smooth.frame, bf.maxit, : lv
## too small.  (Discovered by lowesd)
```

```
## Warning in lo.wam(x, z, wz, fit$smooth, which, fit$smooth.frame, bf.maxit, :
## liv too small.  (Discovered by lowesd)
```

```
## Warning in lo.wam(x, z, wz, fit$smooth, which, fit$smooth.frame, bf.maxit, : lv
## too small.  (Discovered by lowesd)
```

```
plot(gam.lo.i) # plot the resulting two=dimensional surface, akima library is needed here
```



```
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
nFolds <- 10
myFolds <- createFolds(seq(nrow(Wage)),nFolds)

pred <- rep(NA,nrow(Wage))
for (f in seq(nFolds)){
  gam.m2=gam(wage~year+s(age,5)+education,data=Wage[-myFolds[f][[1]],])
  pred[myFolds[f][[1]]] <- predict(gam.m2,Wage[myFolds[f][[1]],])
}
myRMSE(pred,Wage$wage) # the cross-validated RMSE of the model based on year, age and education
```

```
## [1] 35.2027
```

Further questions to answer (homework):

1. Is there any way to further improve the model tested in the last chunk using only these three predictors?
2. Have a look at other predictors. What treatment would you recommend for them?
3. Which non-linear model would you recommend for wage prediction considering all the predictors? Show a model that improves the gam model tested in the last chunk.